PROJECT REPORT (CSN-452)

A report submitted in partial fulfilment of the requirement for the course

DevOps

Part of the degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING



Submitted to

Mrs. Suchi Johari

Submitted by:

Name-Hemant Sharma, Sakshi Patel

SAP ID-
1000020376,100020358

Section - O(P1)

SCHOOL OF COMPUTING

DIT UNIVERSITY, DEHRADUN

(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)

Mussoorie Diversion Road, Dehradun, Uttarakhand - 248009, India.

Declaration

We hereby declare that the project report entitled "**Comparative Study of Manual and Automated Software Deployment**", submitted in partial fullfilment of the requirements of the Bachelor of Technology programm at DIT University for the course CSN452 – DevOps, is based on the work carried out by us.

The practical work described in this report demonstrates a manual deployment process and a Jenkins-based automated deployment pipeline for a Node.js application. The report has been prepared for academic purposes under the guidance of Mrs. Suchi Johari.

Name of the Student Hemant Sharma Signature and Date: *Hemant Sharma*

Name of the Student Sakshi Patel Signature and Date: *Sakshi Patel*

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mrs. Suchi Johari, for her guidance and support during the completion of this DevOps project. Her direction helped us understand the practical aspects of software deployment, automation, and continuous integration.

We also thank the School of Computing, DIT University, for providing the academic environment and resources required to complete this work. Finally, we appreciate the cooperation and contribution of every team member throughout planning, implementation, testing, and documentation.

Hemant Sharma -23A04D0413

Sakshi Patel -23A04D0407

TABLE OF CONTENT

Declaration	i
Acknowledgement	ii
Chapter 1 – Introduction	1
1.1 Introduction	1
1.2 Objectives	2
1.3 Project Description	3–4
Chapter 2 – System Design	5
2.1 System Architecture	5
2.2 Workflow	6
2.3 Module Description	7–8
Chapter 3 – Implementation Modules	9
3.1 Tools and Technologies	9–10
3.2 Work Plan	11
Chapter 4 – Results and Discussions	12–16
4.1 Output Screenshots	12–16
Chapter 5 – Conclusion	17

Chapter 1

1. Introduction

Software deployment is the process of making an application available for use in a target environment. It commonly includes obtaining the source code, installing dependencies, configuring the runtime environment, starting the application, and checking that the application is responding correctly.

In a manual deployment process, a developer or operator performs these activities by executing commands and checking the result. Although this approach is suitable for small applications and learning environments, it can require repeated effort and may lead to inconsistent steps or human error.

Automated deployment uses tools and scripts to perform deployment tasks in a repeatable manner. In this project, Jenkins is used to create a pipeline that checks out the application source code from GitHub, installs Node.js dependencies, starts the application, and verifies the deployment through a health endpoint.

The project compares manual and automated deployment using a small Node.js and Express application. The purpose is to observe the workflow, execution effort, repeatability, and verification process of both approaches in a local development environment.

1.2 Objectives

1. To understand the basic concepts and stages involved in software deployment.
2. To deploy a Node.js application manually using terminal commands.
3. To configure Jenkins to retrieve source code from a GitHub repository.
4. To create a Jenkins pipeline with dependency installation, application deployment, and health verification stages.
5. To compare manual and automated deployment in terms of steps, human effort, consistency, and execution time.
6. To identify common deployment issues and understand how they can be resolved

1.3 Project Description

The project uses a Node.js application stored in a GitHub repository named deployment-comparison. The application is served by an Express server and exposes a health endpoint at /health. Jenkins is configured with a Pipeline job that reads the Jenkinsfile from the repository.

The automated pipeline contains three main stages:

- Install Dependencies: changes to the application directory and runs npm install.
- Deploy Application: starts server.js as a background process using nohup.
- Verify Deployment: sends an HTTP request to http://localhost:3000/health and retries a limited number of times before reporting success or failure.

For the manual experiment, the same application is started directly from the terminal by installing dependencies and running the server. The same application URL and health endpoint are used to verify that it is running. This keeps the application constant while the deployment method changes.

Chapter 2

System Design

The solution uses a simple source-control and deployment architecture. GitHub stores the project files and Jenkins retrieves the selected branch. Jenkins runs the pipeline on the local machine where the Jenkins service is installed. The pipeline uses the local Node.js and npm installation to install dependencies and start the Express server.

Component	Responsibility
Developer / Student	Maintains the application code and commits changes to GitHub.
GitHub Repository	Stores server.js, package.json, public files, and Jenkinsfile.
Jenkins	Checks out the repository and executes the pipeline stages.
Node.js and npm	Provide the JavaScript runtime and dependency management.
Express Application	Serves the web application and health-check endpoint.
Browser / curl	Checks the application URL and verifies the health response.

High-level flow: Developer → GitHub Repository → Jenkins Pipeline → npm install → Node.js Application → Health Check.

2.2 Workflow:

1. Application preparation: create the Node.js project and confirm that server.js starts the application on the configured port.
2. Source control: commit the project files and Jenkinsfile to the main branch of GitHub.
3. Jenkins configuration: create a Pipeline job, select Pipeline script from SCM, choose Git, enter the repository URL, set the branch to */main, and specify the Jenkinsfile path.
4. Manual deployment: open the project directory, install dependencies if required, and start the server using Node.js.
5. Automated deployment: select Build Now in Jenkins. Jenkins checks out the repository and runs the stages defined in the Jenkinsfile.
6. Verification: open the application in a browser and request the /health endpoint to confirm that the service is responding.

•

2.3 Module Descriptions:

1. Source Code Management Module: Uses Git and GitHub to maintain application files and track changes. Jenkins retrieves the selected branch from the remote repository.

2. Jenkins Configuration Module: Defines the pipeline job, repository URL, branch, and location of the Jenkinsfile.

3. Dependency Installation Module: Runs npm install in the application directory to install packages listed in package.json.

4. Deployment Module: Starts the Node.js server as a background process so that the pipeline can proceed to verification.

5. Health Verification Module: Uses curl to send a request to the health endpoint. The stage retries briefly and fails if the endpoint does not respond successfully.

6. Comparison and Reporting Module: Records the steps and observations from manual and automated deployment for comparison.

Chapter 3

Implementation

Modules

3.1 Tools and Technologies:

Tool / Technology	Purpose in the project
Node.js	JavaScript runtime used to execute the backend server.
Express.js	Framework used to create the HTTP server and health endpoint.
npm	Installs and manages project dependencies.
Git	Tracks source-code changes and commits.
GitHub	Hosts the source-code repository.
Jenkins	Runs the automated deployment pipeline.
Groovy / Jenkinsfile	Defines pipeline stages and shell commands.
macOS Terminal	Used for manual deployment and local verification.
Visual Studio Code	Used to edit application files and the Jenkinsfile.
Web Browser and curl	Used to verify application and health endpoint responses.

3.2 Work Plan

Phase 1 – Planning and Requirement Analysis: Identify the application, deployment steps, comparison parameters, and required tools.

Phase 2 – Application Setup: Prepare the Node.js application, package configuration, public files, and health endpoint.

Phase 3 – Manual Deployment: Install dependencies, start the application manually, and verify its response.

Phase 4 – Version Control Setup: Create the GitHub repository and push the project files to the main branch.

Phase 5 – Jenkins Pipeline Setup: Configure Jenkins to use Pipeline script from SCM and locate the Jenkinsfile in the repository.

Phase 6 – Automated Deployment and Testing: Execute the pipeline, resolve configuration issues, and verify the application using the health endpoint.

Phase 7 – Comparison and Documentation: Record observations, compare the two deployment methods, and prepare the report.

3.3 Jenkins Pipeline Implementation

The Jenkinsfile defines a declarative pipeline with three stages. The PATH environment is configured in the relevant stages so Jenkins can locate the Node.js and npm executables installed at /opt/homebrew/bin on the project machine. The application files are located in the deployment-comparison directory within the checked-out repository.

Code:

```
pipeline {
    agent any
    stages {
        stage('Install Dependencies') {
            environment {
                PATH = "/opt/homebrew/bin:${env.PATH}"
            }
            steps {
                dir('deployment-comparison') {
                    sh '''
                        node -v
                        npm -v
                        npm install
                    '''
                }
            }
        }
        stage('Deploy Application') {
            environment {
                PATH = "/opt/homebrew/bin:${env.PATH}"
            }
            steps {
                dir('deployment-comparison') {
                    sh ‘"
```

```
                nohup node server.js > app.log 2>&1 &
            '''
        }
    }
}
stage('Verify Deployment') {
    steps {
        sh '''
            for i in 1 2 3 4 5; do
                if curl -fsS http://localhost:3000/health; then
                    echo "Deployment successful!"
                    exit 0
                fi
                sleep 2
            done
            echo "Deployment failed!"
            exit 1
        '''
    }
}
}
}
```

Chapter 6

Results and Discussions

4.1 Manual Deployment Results

In the manual method, deployment commands are entered directly in the terminal from the application directory. The operator is responsible for selecting the correct directory, installing dependencies, starting the server, and checking the health endpoint.

Commands used for a typical manual run:

```
cd deployment-comparison
npm install
node server.js
```

The application is then checked in a browser at http://localhost:3000 and through the health endpoint at http://localhost:3000/health. Insert the actual terminal and browser screenshots captured during the manual experiment.

4.2 Automated Deployment Results

In the automated method, Jenkins reads the Jenkinsfile from the GitHub repository and executes the pipeline. The successful Jenkins Build #3 displayed a green success status. The recorded build duration visible in Jenkins was 8.4 seconds for that run. This value represents the observed Jenkins build duration in the local test environment and may vary between runs.

The pipeline reached the dependency installation, deployment, and health verification stages. The successful build indicates that Jenkins was able to locate the Jenkinsfile, access Node.js and npm through the configured PATH, and complete the defined pipeline.

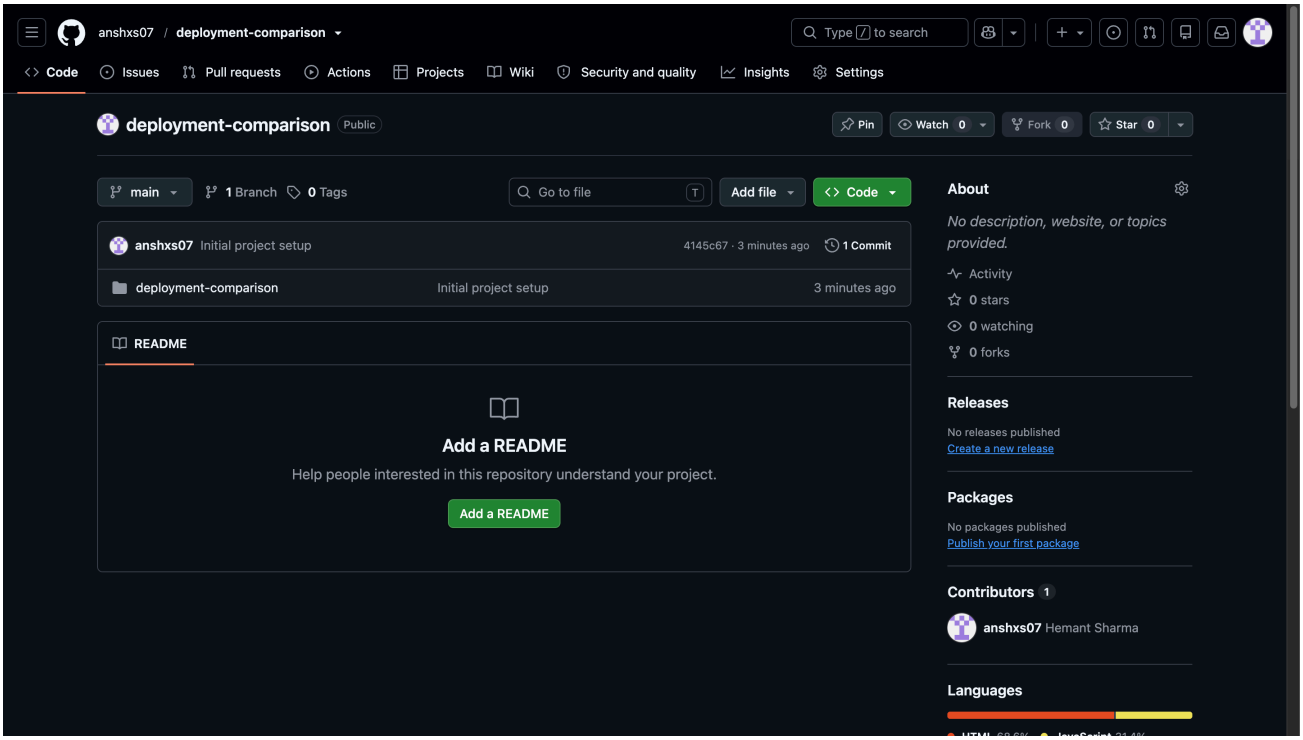
4.3 Comparative Analysis

The comparison below summarizes the expected operational differences. Actual manual deployment time and other measurements should be entered from the experiment rather than estimated.

Parameter	Manual Deployment	Automated Deployment
Execution	Commands are entered by the operator.	Jenkins executes the steps from the Jenkinsfile.
Dependency installation	Operator runs npm install.	Pipeline runs npm install automatically.
Application start	Operator runs node server.js.	Pipeline starts server.js using nohup.
Verification	Operator opens the URL or runs curl.	Pipeline checks the /health endpoint.
Repeatability	Depends on the operator following the same steps.	Defined steps are repeated through the pipeline.
Human effort	Requires manual command execution.	Requires initiating or triggering the Jenkins job.
Observed duration	Record measured value: _____ seconds.	Build #3 displayed 8.4 seconds in Jenkins.

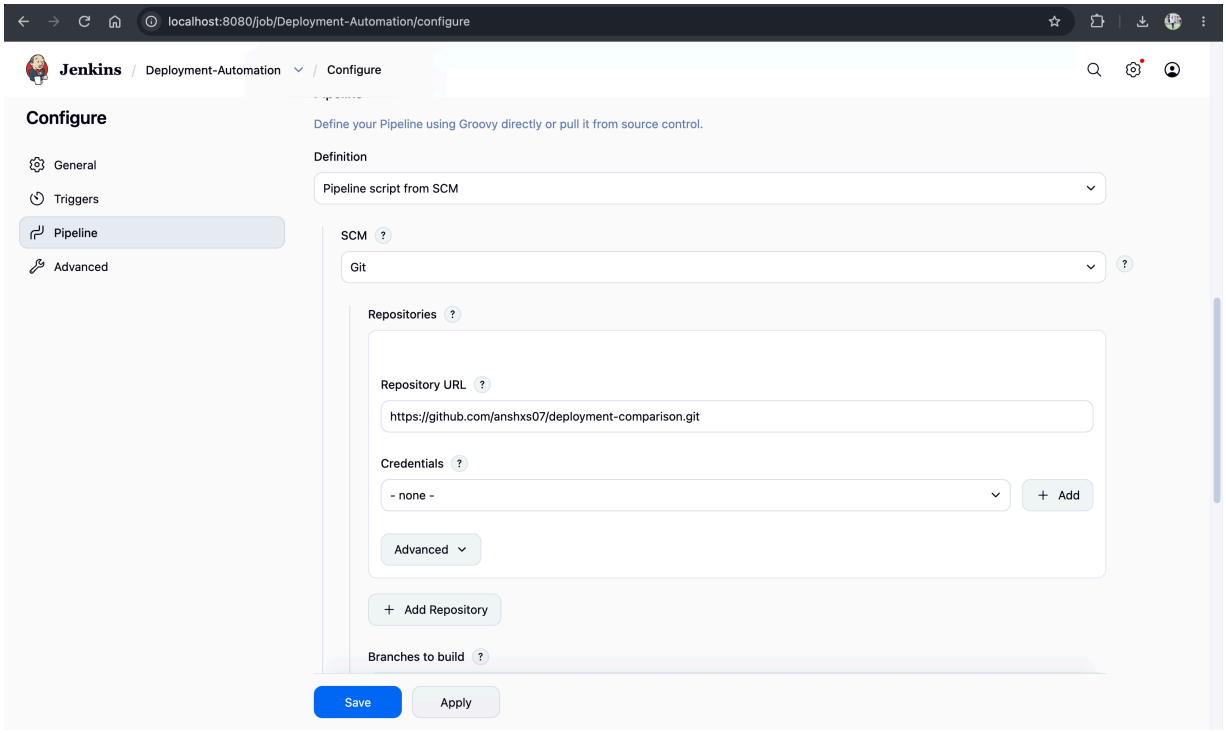
4.4 Output Screenshots

Figure 4.1 – GitHub Repository



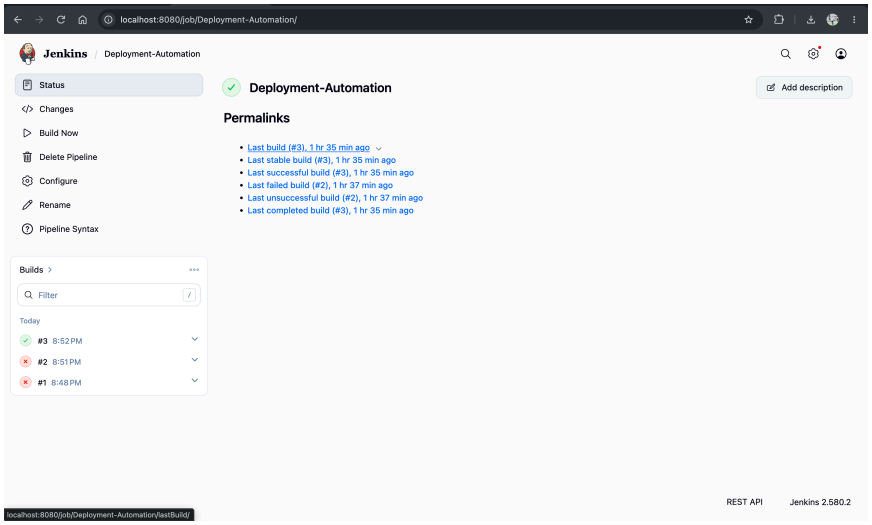
Screenshot showing the project files and Jenkinsfile in the GitHub repository.

Figure 4.2 – Jenkins Pipeline Configuration



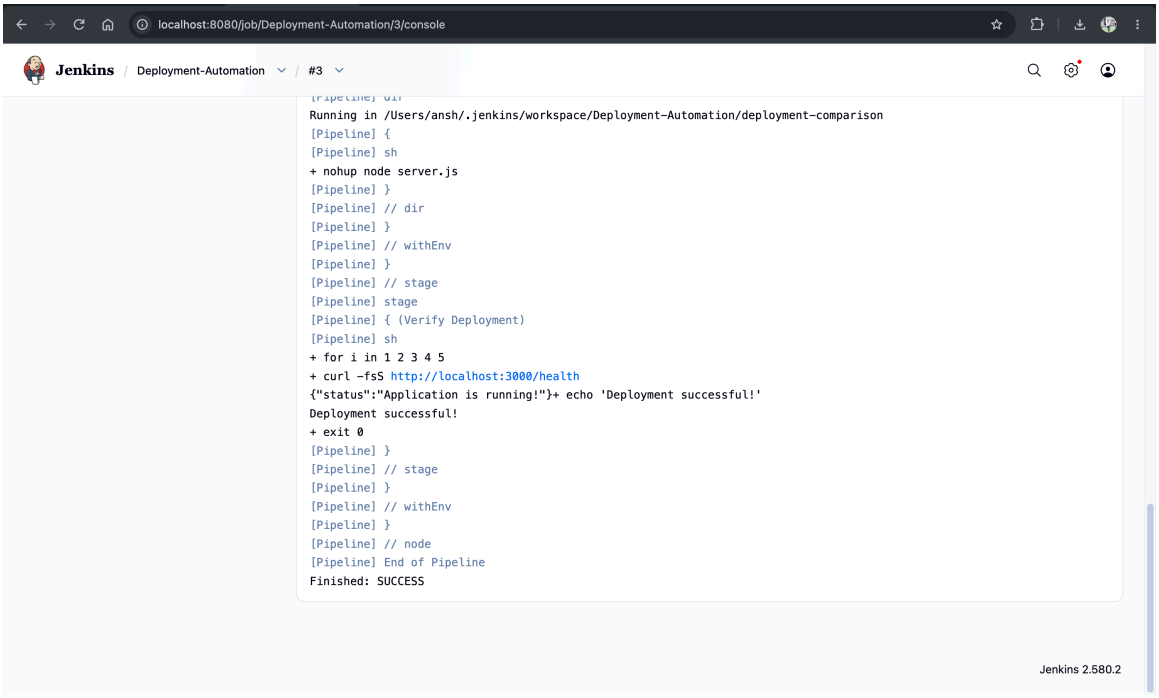
Screenshot showing Pipeline script from SCM, Git repository URL, branch, and script path.

Figure 4.3 – Jenkins Successful Build



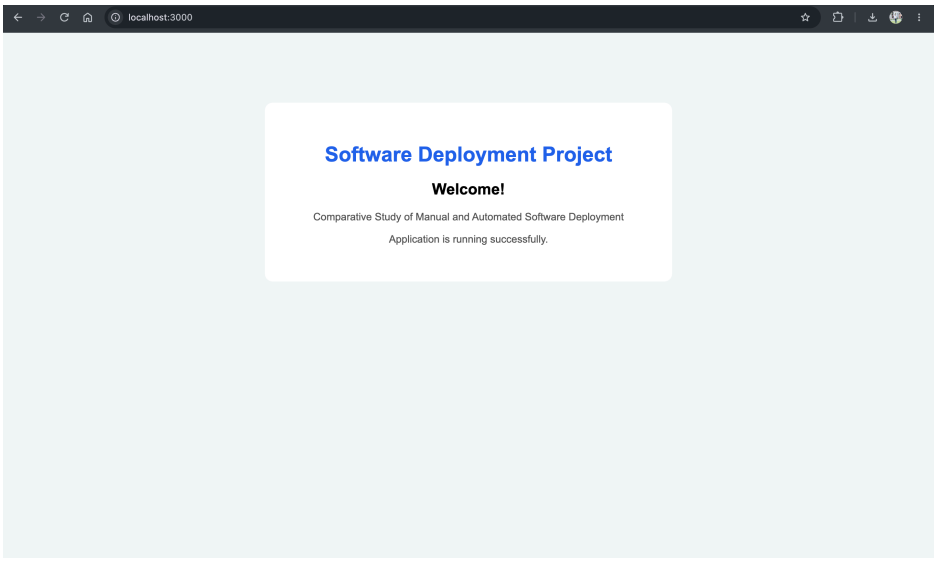
Screenshot of Build #3 with the green success indicator.

Figure 4.4 – Jenkins Console Output



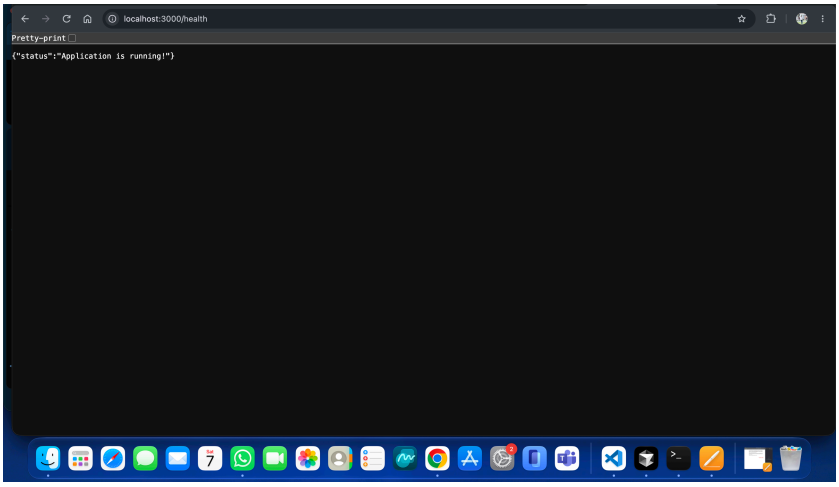
Screenshot showing the successful execution of the pipeline stages.

Figure 4.5 – Application Running



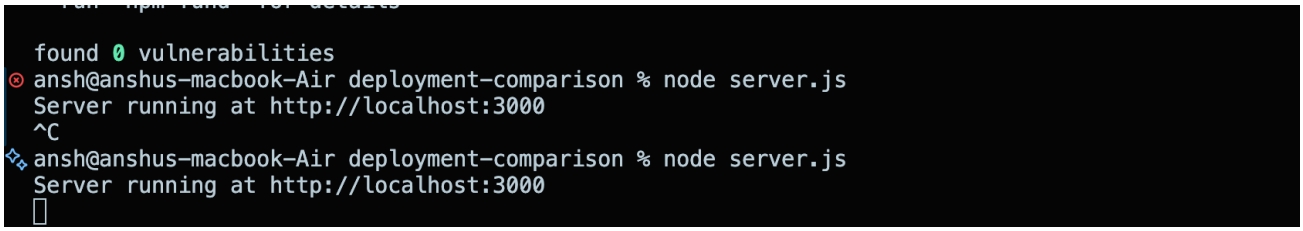
Browser screenshot of the application at http://localhost:3000.

Figure 4.6 – Health Check



Browser or terminal screenshot showing the successful /health response.

Figure 4.7 – Manual Deployment



Terminal screenshot showing manual dependency installation and server startup.

Chapter 5 Conclusion

The project, “Comparative Study of Manual and Automated Software Deployment,” demonstrates two approaches for deploying the same Node.js application. In manual deployment, the operator performs dependency installation, server startup, and verification through terminal commands. In automated deployment, Jenkins retrieves the source code from GitHub and executes the deployment steps defined in a Jenkinsfile.

The successful Jenkins Build #3 demonstrates that the configured pipeline can complete the defined process in the local environment. The project also provided practical experience with GitHub integration, Jenkins Pipeline configuration, Node.js environment variables, shell commands, and HTTP-based health verification.

The comparison shows that manual deployment is simple for small tasks, while automated deployment improves repeatability and reduces the need to re-enter deployment commands. The project therefore illustrates the value of automation as application deployment becomes more frequent or complex.

Future Scope

- Configure a GitHub webhook to trigger the Jenkins pipeline automatically whenever code is pushed.
- Add automated unit and integration tests before deployment.
- Build and package the application as a Docker image.
- Deploy the application to a separate staging or production environment rather than only the local machine.
- Add deployment logs, notifications, and monitoring dashboards.
- Implement rollback to the last successful application version if deployment verification fails.
- Use Jenkins credentials and secure environment variables for applications that require secrets.